

Learning on Graphs

Homework 1: Graph Operators, Message Passing, and the Limits of Depth

General instructions

This homework studies the foundations of graph learning through a mixture of theory, implementation, experiments, and short explanations. The assignment covers graph representations, graph operators, spectral smoothing, semi-supervised node classification, oversmoothing diagnostics, and the 1-WL expressivity limit.

- Submission is individual.
- Use Python, NumPy, NetworkX, PyTorch, PyTorch Geometric, and Matplotlib.
- Submit one PDF report and one runnable notebook or Python package.
- Fix all random seeds and include clear run instructions.
- Your report should be concise. A good target is 5 to 8 pages.
- Do not paste large code blocks into the report. Include only the important equations, plots, tables, and explanations.

Deliverables

Submit a single zip file containing:

1. `report.pdf`
2. `hw1.ipynb` or `main.py` with helper files
3. `requirements.txt` or environment notes
4. All figures used in the report

Your code must run from a clean restart. If a result is stochastic, report the seed and explain the variability.

Point allocation

Component	Points
Part A: Theory warm-up	20
Part B: NetworkX graph operators	20
Part C: Semi-supervised node classification with PyG	30
Part D: Depth and oversmoothing	20
Part E: 1-WL and expressivity toy study	10
Bonus: Oversquashing intuition on a barbell graph	10

Learning goals

By the end of this homework, you should be able to:

1. Represent graphs using adjacency matrices, sparse edge lists, and PyG `Data` objects.
2. Compute graph statistics, graph Laplacians, and simple graph signal operations.
3. Explain permutation equivariance and permutation invariance.
4. Train basic GNN models for node classification.
5. Diagnose oversmoothing using depth sweeps, Dirichlet energy, and embedding distances.
6. Understand why standard message passing is limited by 1-WL, and how extra node information can help.

Graphs used in this homework

This homework intentionally uses three different graph settings.

1. **Zachary's Karate Club graph:** a small, deterministic social-network graph with a built-in node attribute `club`. Use this graph for small-scale inspection, graph statistics, matrix construction, and graph-signal smoothing.
2. **Matched Erdos-Renyi graphs:** random control graphs generated with the same number of nodes and approximately the same edge density as Karate Club. Use them to compare structure against a simple random baseline.
3. **Cora:** a citation network from PyTorch Geometric's `Planetoid` benchmark. Use it for Parts C and D on node classification and oversmoothing diagnostics.

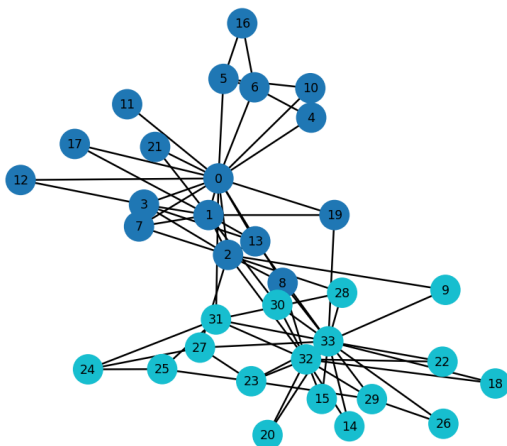


Figure 1: Karate Club. A fixed small graph with an interpretable community split.

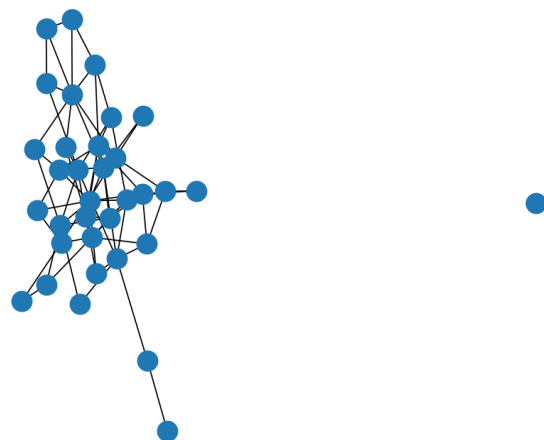


Figure 2: One example matched Erdos-Renyi control graph.

Starter code for loading the homework graphs

The first notebook cell installs PyTorch Geometric by default:

```
%pip install -q torch-geometric
```

The notebooks already contain these loading commands. They are shown here so the assignment is self-contained. The notebooks provide graph and data loading, but the computational helper functions are not provided to students. You must implement the graph statistics, adjacency and Laplacian construction, normalized GCN matrix construction, Dirichlet energy, and pairwise-distance diagnostics yourself.

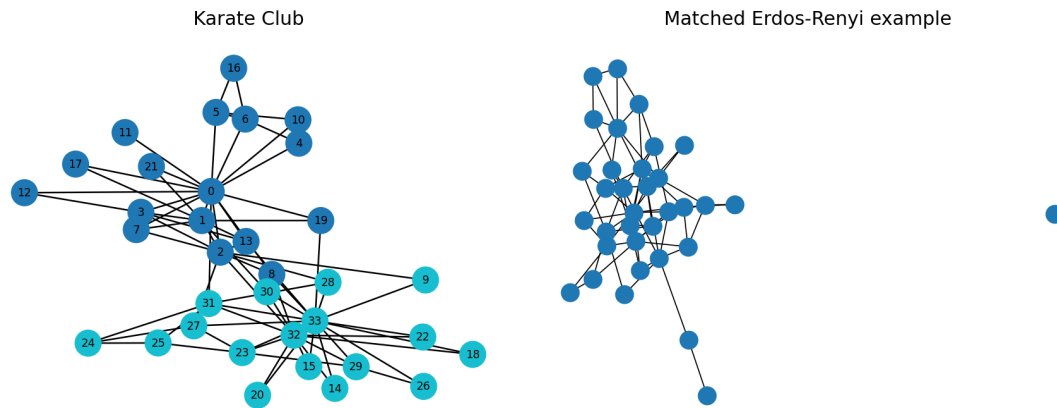


Figure 3: Illustrative views of two of the graphs used in this homework: the Karate Club graph and one matched Erdos-Renyi example. Students should also visualize these graphs in the notebook.

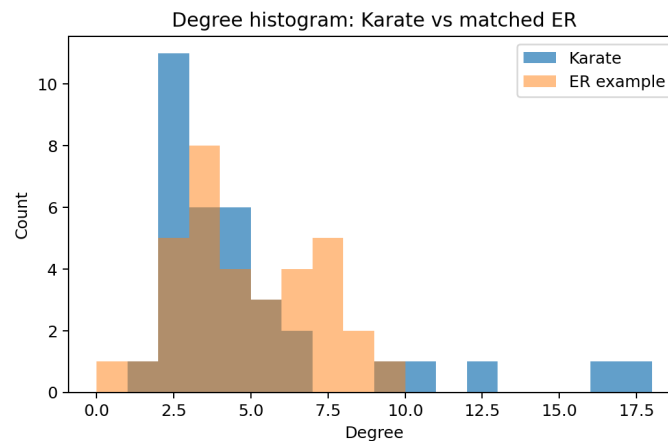


Figure 4: Illustrative degree histogram comparison between the Karate Club graph and a matched Erdos-Renyi control.

Karate Club and matched Erdos-Renyi controls.

```
import numpy as np
import networkx as nx

G_karate = nx.karate_club_graph()
karate_club_labels = nx.get_node_attributes(G_karate, "club")
karate_label_to_id = {"Mr. Hi": 0, "Officer": 1}
karate_y = np.array([karate_label_to_id[karate_club_labels[i]]
                      for i in sorted(G_karate.nodes())])
```

```
n = G_karate.number_of_nodes()
m = G_karate.number_of_edges()
p_match = 2 * m / (n * (n - 1))
G_er_1 = nx.erdos_renyi_graph(n, p_match, seed=1)
G_er_2 = nx.erdos_renyi_graph(n, p_match, seed=2)
G_er_3 = nx.erdos_renyi_graph(n, p_match, seed=3)
```

Cora through PyTorch Geometric.

```
from torch_geometric.datasets import Planetoid

dataset = Planetoid(root="data/Planetoid", name="Cora")
data = dataset[0]
print(data)
print(dataset.num_features, dataset.num_classes)
```

In your report, include the required figures listed in the checklist: Karate Club, at least one matched Erdos-Renyi graph, degree histograms, Cora training curves, oversmoothing visualizations, toy graphs, and the bonus graphs if you attempt the bonus.

You need to implement: `graph_stats`, `adjacency_matrix_np`, `laplacian_np`, `normalized_adjacency_np`, `symmetric_gcn_matrix_nx`, `symmetric_gcn_matrix_from_edge_index`, `dirichlet_energy`, and `mean_pairwise_di`.

1 Part A: Theory warm-up, 20 points

A1. Permutation equivariance, 6 points

Let $A \in \mathbb{R}^{n \times n}$ be an adjacency matrix, $X \in \mathbb{R}^{n \times d}$ be node features, and P be a permutation matrix. Consider the GCN-like linear layer

$$H = \tilde{A}XW,$$

where $W \in \mathbb{R}^{d \times h}$ and $\tilde{A}$ is a normalized adjacency matrix.

Show that if the graph and node features are relabeled as

$$A' = PAP^\top, \quad X' = PX,$$

then the output satisfies

$$H' = PH.$$

State clearly what assumption you need on the normalization $\tilde{A}$.

A2. Laplacian and Dirichlet energy, 5 points

For an undirected graph with unnormalized Laplacian $L = D - A$, prove that

$$x^\top Lx = \frac{1}{2} \sum_{(i,j) \in E} (x_i - x_j)^2.$$

Then explain in 2 to 3 sentences why small Dirichlet energy corresponds to a smooth graph signal.

A3. Locality of polynomial filters, 4 points

Let

$$p_K(L)x = \sum_{k=0}^K \theta_k L^k x.$$

Explain why $(p_K(L)x)_i$ depends only on the K -hop neighborhood of node i .

A4. Aggregation and multiplicity, 5 points

Consider the two multisets

$$M_1 = \{\{a, b\}\}, \quad M_2 = \{\{a, a, b, b\}\}.$$

Answer the following:

1. Show that mean aggregation maps them to the same value.
2. Explain why max aggregation can also lose multiplicity information.
3. Explain why sum aggregation is the natural standard choice when trying to match the 1-WL ceiling.

2 Part B: NetworkX graph operators, 20 points

In this part, you will work with two kinds of small graphs:

1. Zachary's Karate Club graph, loaded with `nx.karate_club_graph()`.
2. Erdos-Renyi random graphs generated with `nx.erdos_renyi_graph(n, p, seed)`.

Use Karate Club as the main fixed graph. Use the three provided Erdos-Renyi graphs as random controls. Use the loading code provided above and in the notebook.

B1. Load and inspect Karate Club, 4 points

Load the Karate Club graph. Report:

1. Number of nodes.
2. Number of edges.
3. Average degree.
4. Density.
5. Number of connected components.
6. The two club labels and the number of nodes in each club.

Visualize the graph with nodes colored by their `club` attribute.

B2. Generate comparable Erdos-Renyi graphs, 5 points

Let n be the number of nodes in Karate Club, and let m be its number of edges. Set

$$p_{\text{match}} = \frac{2m}{n(n-1)}.$$

Generate three Erdos-Renyi graphs with the same n and p_{match} using three different seeds.

For each graph, report the same statistics as in B1, excluding club labels. Also include at least one ER graph visualization and a degree histogram figure in both the notebook and the PDF report. In your report, answer:

1. Are the Erdos-Renyi graphs always connected?
2. How different are their degree distributions from Karate Club?
3. Why is the fixed Karate graph useful for reproducibility, while Erdos-Renyi graphs are useful for controlled variation?

B3. Matrix representations, 4 points

Implement the required matrix-construction routines yourself. For the Karate graph, construct:

1. The adjacency matrix A .
2. The degree matrix D .
3. The unnormalized Laplacian $L = D - A$.
4. The normalized adjacency with self-loops

$$\tilde{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}.$$

Report the shapes and number of nonzero entries of each matrix. Explain why dense adjacency matrices are not suitable for large sparse graphs.

B4. Graph signal smoothing, 7 points

Use the provided Karate labels `karate_y` from the notebook loading cell, where class 0 is `Mr. Hi` and class 1 is `Officer`. Define the scalar signal

$$x_i = 1 - 2\text{karate_y}_i,$$

so class 0 maps to $+1$ and class 1 maps to -1 . Compute

$$x^{(k+1)} = \tilde{A}x^{(k)}$$

for $k = 0, 1, \dots, 20$.

Plot:

1. The signal on the graph for $k = 0, 1, 5, 20$, using the same color range in every panel.
2. The Dirichlet energy $(x^{(k)})^\top Lx^{(k)}$ as a function of k .
3. The variance of node values as a function of k .

Explain what happens as k increases.

3 Part C: Semi-supervised node classification with GCN on Cora, 30 points

In this part, you will train exactly two models on the Cora citation network using PyTorch Geometric:

1. An MLP baseline that ignores graph edges.
2. A two-layer GCN using `GCNConv`.

Use the provided notebook cell to load Cora with

```
Planetoid(root="data/Planetoid", name="Cora").
```

Train both models for exactly 500 epochs. Select the checkpoint with the best validation accuracy and report the corresponding test accuracy.

C1. Dataset inspection, 4 points

Report:

1. Number of nodes.
2. Number of edges.
3. Number of node features.
4. Number of classes.
5. Number of training, validation, and test nodes.
6. Why this is a transductive node-classification setup.

C2. MLP baseline, 8 points

Implement an MLP with one hidden layer:

$$X \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}.$$

Use hidden dimension 64 and dropout 0.5. Train for 500 epochs.

Report the architecture, number of parameters, best validation accuracy, test accuracy at the best validation checkpoint, and training curves for loss and validation accuracy.

C3. Two-layer GCN, 10 points

Implement a two-layer GCN using `GCNConv`:

$$X \rightarrow \text{GCNConv} \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{GCNConv}.$$

Use hidden dimension 64 and dropout 0.5. Train for 500 epochs with the same optimizer settings as the MLP.

Report the architecture, number of parameters, best validation accuracy, test accuracy at the best validation checkpoint, and training curves for loss and validation accuracy.

C4. Results table and discussion, 8 points

Provide a table of the following form:

Model	Parameters	Best val. acc.	Test acc.	Notes
MLP				
GCN				

Discuss the results in 5 to 8 sentences. Focus on what graph information the GCN uses that the MLP ignores, whether the GCN improves over the MLP, and whether the curves suggest overfitting.

4 Part D: Depth and oversmoothing with fixed GCN propagation, 20 points

This part uses a non-trainable GCN propagation operator. There are no learnable weights, no optimizer, and no training in Part D. Use the symmetric GCN matrix

$$S = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}, \quad \hat{A} = A + I.$$

Starting from node features, compute

$$H^{(k+1)} = SH^{(k)}.$$

The goal is to isolate the smoothing effect of repeated GCN propagation.

D1. Fixed GCN propagation on Cora, 8 points

Implement the symmetric GCN matrix and diagnostic functions yourself. Use the Cora node features as $H^{(0)}$. Propagate with the fixed GCN matrix for depths

$$k \in \{0, 1, 2, 4, 8, 16, 32, 64\}.$$

For every listed depth, report:

1. Dirichlet energy $\text{Tr}((H^{(k)})^\top L H^{(k)})$.
2. Mean pairwise node-feature distance.

D2. Cora smoothing diagnostic plots, 5 points

Plot the two quantities from D1 as a function of propagation depth. Explain how the curves change as depth grows, and what this says about oversmoothing.

D3. Provided ER oversmoothing visualization graph, 5 points

Use the provided graph and signal in the notebook. The graph is a connected Erdos-Renyi graph of decent size. The graph and initial signal are provided, but you must implement the GCN propagation matrix and diagnostics yourself.

The notebook provides:

```
G_over, G_over_seed = make_connected_er_graph(n=120, p=0.06, seed=7)
rng_over = np.random.default_rng(0)
over_signal0 = rng_over.normal(size=G_over.number_of_nodes())
over_signal0 = over_signal0 / np.max(np.abs(over_signal0))
```

Build the symmetric GCN matrix S for this graph and propagate

$$h^{(k+1)} = Sh^{(k)}$$

for depths

$$0, 1, 2, 4, 8, 16, 32, 64.$$

Plot Dirichlet energy, variance, and mean pairwise distance versus depth. Also visualize the graph signal at all listed depths using the same color range in every panel, with `vmin=-1` and `vmax=1`.

D4. Short reflection, 2 points

Write a short paragraph explaining why repeated GCN propagation can help nodes communicate over longer graph distances, but can also make node representations less discriminative.

Connected ER graph for oversmoothing: $n=120$, $p=0.06$, seed=7

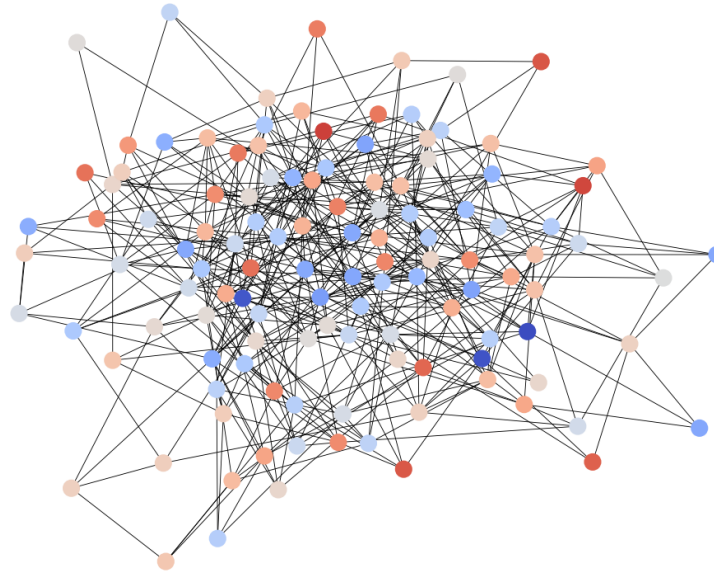


Figure 5: Provided connected Erdos-Renyi graph for the oversmoothing visualization, colored by the initial signal.

5 Part E: 1-WL and expressivity toy study, 10 points

The graphs used in this part are shown in Figure 6. You should also visualize them in your notebook.

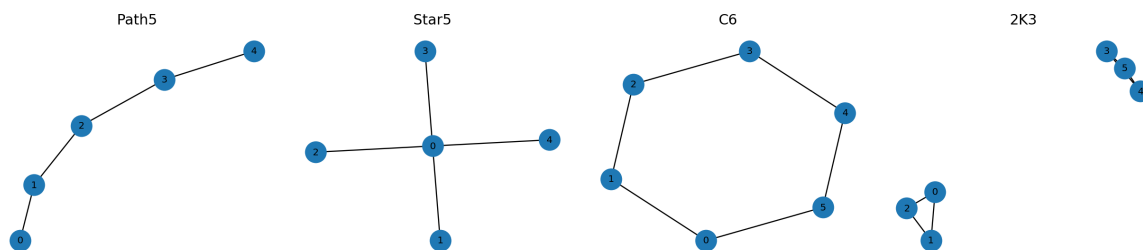


Figure 6: Toy graphs for the 1-WL study. Path5 and Star5 are a same-size warmup pair with 5 nodes each. C_6 and $2K_3$ are the same-size failure pair with 6 nodes each.

E1. Implement 1-WL color refinement, 3 points

Implement the 1-WL color refinement algorithm. Your function should take a NetworkX graph and return the color histogram after each iteration.

Test it on:

1. A path graph with 5 nodes.
2. A star graph with 5 nodes.

3. The cycle graph C_6 .
4. The disjoint union of two triangles, $2K_3$.

Path5 and Star5 are warmup examples. They both have 5 nodes. The main failure case is C_6 versus $2K_3$, which both have 6 nodes. Do not compare a 5-node graph to a 6-node graph as an isomorphism test.

Report the color histograms for at least 3 iterations, and visualize the histogram evolution in the notebook.

E2. Failure case: C_6 versus $2K_3$, 2 points

Show that 1-WL with constant initial node labels fails to distinguish C_6 and $2K_3$. These graphs have the same number of nodes and the same degree at every node, so the failure is not caused by graph size. Explain why every node receives the same update at every iteration.

E3. Structural augmentation with RWSE, 3 points

Use RWSE structural encodings as initial node information on C_6 and $2K_3$.

Discuss:

1. Does RWSE help distinguish the graphs?
2. Why is RWSE more principled than arbitrary node IDs?
3. Does RWSE preserve permutation equivariance or invariance?

E4. Node coloring or numbering as a contrast case, 2 points

Also initialize C_6 and $2K_3$ with unique node identities, for example by giving each node its own color or its own numeric ID. Visualize the graphs with this node coloring in the notebook and include a small figure or discussion in the PDF.

Report what happens to the 1-WL histograms, and explain why this can trivially distinguish the graphs. Then explain why this is not a permutation-invariant or structurally meaningful augmentation in general.

6 Bonus: Oversquashing intuition on a barbell graph, 10 points

Use the provided graphs from the notebook:

```
G_barbell = nx.barbell_graph(8, 4)
G_shortcut = G_barbell.copy()
G_shortcut.add_edge(0, max(G_shortcut.nodes()))
```

The two graphs are shown in Figure 7. You should also visualize them in your notebook.

Bonus 1. Bottleneck statistics, 3 points

For both graphs, compute and report:

1. Diameter.
2. Average shortest path length.
3. Spectral gap λ_2 of the normalized Laplacian.
4. Maximum edge betweenness centrality and the edge attaining it.

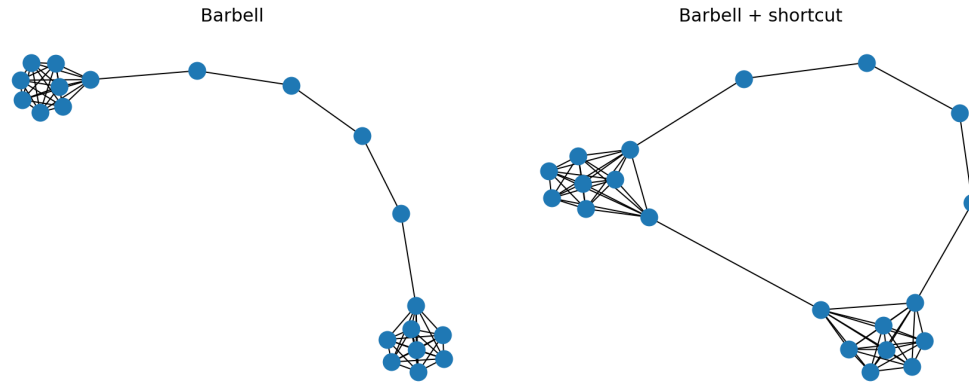


Figure 7: Barbell graph before and after adding a shortcut edge.

Bonus 2. Signal transfer experiment, 4 points

Inject a scalar signal at source node 0:

$$h_0 = e_0.$$

Propagate with the symmetric GCN matrix

$$h^{(k+1)} = Sh^{(k)}, \quad S = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}.$$

For both graphs, plot over propagation steps:

1. Mean absolute signal on the right clique nodes.
2. Mean absolute signal on the bridge/path nodes.

Use right clique nodes 12:20 and bridge/path nodes 8:12, as in the notebook.

Bonus 3. Smoothing and snapshots, 3 points

For both graphs, also plot the global variance of $h^{(k)}$. Then visualize signal snapshots at steps

$$0, 1, 4, 16, 40$$

using the same color range for all panels. Explain how the shortcut changes both signal transfer and smoothing.

Suggested sanity checks

Before submitting, verify the following:

- Your notebook runs from top to bottom.
- The same random seeds produce the same reported numbers.
- All requested plots and tables from Parts B-E appear in the PDF report.
- Each plot has a title, labeled axes, and a caption in the report.
- Multi-panel smoothing or propagation figures use a shared color range.
- Your code distinguishes dense matrices from sparse graph storage.
- Your Cora experiment uses the official train, validation, and test masks.